

Security Log Monitoring & Suspicious Activity Detection System

Project Report
Language: Python 3

1. Introduction

Every system that has a login page also has people trying to get in where they shouldn't. Sometimes it is a forgetful user typing the wrong password, and sometimes it is an attacker guessing passwords over and over. Servers record every one of these attempts in log files, but reading through hundreds of lines by hand is slow and it is easy to miss the important ones.

This project is a small Python program that does that first pass automatically. It reads a set of login records, counts the failed attempts for each user, flags the entries that look suspicious, and saves a short report. It is a simple, rule-based tool and is meant to show how basic programming ideas can be applied to a real cybersecurity problem.

2. Aim and Objectives

Aim: To build a basic Python program that checks security logs and detects suspicious login activity.

Objectives:

- Read login log entries and separate them into time, username, login status and IP address.
- Count the number of failed login attempts for every user.
- Detect repeated failures (three or more attempts by the same user).
- Detect login attempts made by an unknown user.
- Show clear warnings on screen and give a final status of SAFE or ALERT.
- Save all findings in a text report, `security_report.txt`.

3. Requirements

Type	Requirement
Hardware	Any computer or laptop
Software	Python 3
Editor (any one)	VS Code, IDLE, PyCharm or similar
Libraries	None. The program uses only built-in Python features.

4. How the Program Works

The program stores ten sample log entries in a list. Each entry follows the same format:

```
HH:MM user=<username> login=<success|failed> ip=<address>  
e.g. 10:15 user=itachi login=failed ip=192.168.1.8
```

For every entry the program splits the line at the spaces, and then splits each part at the "=" sign to pull out the actual values. If the login status is *failed*, the user's counter in a dictionary is increased by one. The entry is then checked against two detection rules.

4.1 Detection Rules

Rule	Condition	Why it matters
Repeated failures	A user reaches 3 or more failed logins	Could be a password-guessing (brute-force) attempt
Unknown user	A failed login comes from the user "unknown"	The attempt is not linked to any real account

Any entry that matches a rule is added to a *suspicious* list (duplicates are skipped). At the end, if that list is not empty the final status is **ALERT**; otherwise it is **SAFE**. A user who fails once and then logs in successfully, like *pranjal*, is not treated as suspicious.

5. Algorithm

1. Start.
2. Store the sample login logs in a list.
3. Create an empty dictionary for failed attempts and an empty list for suspicious logs.
4. Take each log entry one by one.
5. Separate it into time, user, status and IP address.
6. If the status is *failed*, add 1 to that user's failed-attempt count.
7. If the count is 3 or more, or the user is "unknown", add the log to the suspicious list.
8. After all logs are read, display the logs, the failed-attempt counts and the warnings.
9. Display the final status (ALERT if anything suspicious was found, otherwise SAFE).
10. Write the same summary to `security_report.txt`.
11. Stop.

6. Output and Testing

The program was run on the sample data. The sample was designed to cover four different situations so that each rule could be tested.

User	What happens in the logs	Expected	Result
harshh	Two successful logins	Not suspicious	Pass
itachi	Three failed logins in a row	Flagged (3 failures)	Pass
pranjal	One failure, then a success	Not suspicious	Pass
unknown	Two failed logins from 45.67.89.10	Flagged (unknown user)	Pass

6.1 Failed login count

User	Failed attempts
itachi	3
pranjal	1
unknown	2

6.2 Suspicious activity detected

```
WARNING: 10:17 user=itachi login=failed ip=192.168.1.8
WARNING: 10:40 user=unknown login=failed ip=45.67.89.10
WARNING: 10:41 user=unknown login=failed ip=45.67.89.10
```

```
FINAL SECURITY STATUS
ALERT! Suspicious activity detected
```

Note that only itachi's *third* failed attempt (10:17) is flagged, because that is the moment the count reaches the threshold. Both attempts by the unknown user are flagged straight away.

7. Limitations

- It works on sample logs stored inside the program, not on a real system.
- The rules are simple. It counts failures across the whole log and does not look at time gaps, so three failures spread over a full day would be treated the same as three in one minute.
- The threshold of 3 is fixed in the code.
- It does not check whether an IP address is trusted or where it is located.

8. Future Scope

- Read logs from a real file or watch a live log in real time.
- Make the failure threshold and time window configurable.
- Flag suspicious IP addresses, not just usernames.
- Add a graphical interface or a dashboard.
- Send automatic alerts by email or message when something suspicious is found.
- Export the report as CSV or JSON, and add unit tests for the detection logic.

9. Conclusion

This project shows that useful security tooling can be built from very basic Python: loops, conditions, string handling, lists and dictionaries. The program correctly separated normal behaviour from suspicious behaviour in the sample data and produced a clear on-screen result and a saved report. It is a small first step, but the same idea of counting, comparing and flagging is the starting point of real log-monitoring tools, and the program can be extended in many of the ways listed above.